Base Info

Project Title

Graph-Based Social Network Analysis for Misinformation Detection

https://github.com/f20230198-art/Graph-Based-Social-Network-Analysis-for-Misinformation-Detection/blob/main/Project_Proposal_Documentation.md

Abstract

The rapid spread of misinformation on social media platforms poses serious social and political risks. Traditional detection systems mainly analyze the textual content of posts, but misinformation often spreads through coordinated social interactions rather than through text alone. This project proposes a graph-based framework that models social media as a network of interacting users to detect misinformation. Users are represented as nodes, and interactions such as reposts, replies, or mentions form connections between them. By extracting network features such as community structures, influence patterns, and information propagation behavior, machine learning models can identify suspicious spreading patterns associated with misinformation campaigns. The system aims to improve detection accuracy by combining content and interaction patterns, enabling earlier identification of coordinated misinformation networks.

Project Idea in Simple Terms

Most fake news detection systems check:

- What is written in the post?

This project instead asks:

- How is the post spreading?
- Who is spreading it?

- Does the spread pattern look suspicious?

Misinformation often spreads through:

- Coordinated groups
- Bot-like accounts
- Highly connected hubs
- Sudden rapid resharing

So we analyze **interaction patterns**, not just text.

Core Concept: Social Media as a Graph

We convert social media data into a network.

Graph Structure

- **Nodes** → Users/accounts
- **Edges** → Interactions between users

Example interactions:

- Retweets or shares
- Mentions
- Replies
- Comments
- Following relationships

Example:
If User A retweets User B, we connect them.

How Misinformation Looks Different in Graphs

Normal news:

- Spreads gradually
- Diverse users share content
- Less clustering

Misinformation:

- Rapid spreading
- Same group repeatedly shares
- Dense suspicious communities
- Few accounts control spread

We detect these structural differences.

Research Papers

Ashmit

1) Advanced Text Analytics - Graph Neural Network for Fake News Detection in Social Media
https://www.arxiv.org/pdf/2502.16157

2) A Graph Network Approach to Disinformation Detection in Social Media
https://www.researchgate.net/publication/391973703_A_Graph_Network_Approach_to_Disinformation_Detection_in_Social_Media

3) Context-Based Fake News Detection using Graph Based Approach: ACOVID-19 Use-case
https://dl.acm.org/doi/10.1145/3729706.3729819

4) Fake news detection: A survey of graph neural network methods
https://pmc.ncbi.nlm.nih.gov/articles/PMC10036155/

5) Misinformation detection based on news dispersion
https://ieeexplore.ieee.org/document/10167997

6) “Unsupervised Fake News Detection: A Graph-based Approach (GTUT)” – Gangireddy et al., conference paper (HT’20 style), 2020
https://pureadmin.qub.ac.uk/ws/files/212663108/ht20_crc.pdf

7) “Fake News Detection on Twitter Using Propagation Patterns” – (Spanakis group), book chapter / conference paper, 2020
https://dke.maastrichtuniversity.nl/jerry.spanakis/wp-content/uploads/2020/11/10.1007@978-3-030-61841-4_MMGWJS.pdf

8) “Analysis of Fake News Detection using Graph Neural Networks” – Hiremath et al., 2023
https://ieeexplore.ieee.org/document/10405304/

9) “Advancing fake news detection with graph neural network and deep learning techniques” – Gul et al., 2025
https://zuscholars.zu.ac.ae/cgi/viewcontent.cgi?article=8253&context=works

Sachin

1) O. G. Orugboh, A. Ezeogu, and O. O. Juba, “A graph theory approach to modeling the spread of health misinformation in aging populations on social media platforms,” *Multidisciplinary Journal of Healthcare (MJH)*, vol. 2, no. 1, Aug. 2025. https://researchcorridor.org/index.php/mjh/article/view/503

2) W. Arshad, A. Manzoor, and M. Hassan, “Fake news detection via textual BERT embeddings and knowledge-aware graph neural networks,” *Multidisciplinary Journal of Healthcare (MJH)*, vol. 1, no. 1, Aug. 2025. https://gajcet.com/index.php/gajcet/article/view/16

3) T. Bellam and S. Aakula, "Contextual and Sentiment-Based Hybrid Models for Detecting Fake News on Social Networks," *2025 IEEE 4th World Conference on Applied Intelligence and Computing (AIC)*, GB Nagar, Gwalior, India, 2025, pp. 323-328, doi: 10.1109/AIC66080.2025.11212091.

https://ieeexplore.ieee.org/abstract/document/11212091?casa_token=2ZvyC58Y16AAAAAA:1fuVdTCHqzdjmWsxWDM67qs5b3bRdz1l2oVcvBwZv82tF5FECvm7iZ1BN2SZmdtxlN-Smp-EK5v9PQ

4)

Research Gaps

Key research gaps and limitations

Based on the GNN-focused survey and recent hybrid papers, several gaps align directly with your intended contributions.

- Static vs dynamic graphs:
  Most GNN fake-news models treat the interaction graph as static, even though misinformation cascades evolve over time; temporal dynamics are often reduced to simple features (e.g., time to first 100 retweets) rather than full dynamic graph modeling.
  There is limited use of temporal GNNs or diffusion-aware architectures that explicitly model the sequence and timing of interactions, especially for cross-event generalization.
- Graph construction choices underexplored:
  Many works build article–user or user–user graphs with heuristic edge definitions (e.g., any retweet/mention) and simple weights, with little ablation on how graph topology choices affect performance.
  Trade-offs between different node types (users vs posts vs publishers), edge semantics (retweet vs reply vs co-engagement), and edge weighting schemes (frequency, recency, rumor-spread rate) remain under-studied.
- Limited multimodal and multi-view integration:
  Classic methods often use either content or network, or concatenate hand-crafted features, whereas recent hybrid GPT-GNN and multimodal GNN models show that explicitly combining text embeddings with graph structure yields sizable gains.
  However, designs for principled fusion (late vs early fusion, attention over modalities) and explainable cross-modal reasoning are still emergent research topics.
- Generalization across topics/events and domains:
  Many fake news models are trained and tested within a single domain (e.g., US politics or COVID-19), which risks overfitting to topical cues; the GNN survey highlights concerns about poor generalization and domain shift.
  Datasets like MiDe22 explicitly span multiple events (COVID-19, Russia–Ukraine war, refugees, etc.), but cross-event and cross-language evaluations are still relatively sparse.
- Label and dataset limitations:
  Labeling often depends on fact-checker platforms (PolitiFact, Snopes, etc.), creating label bias and limited coverage of everyday misinformation.
  Graph datasets with full propagation trees and rich user metadata (e.g., FibVID,

FakeNewsNet) remain relatively few, with platform policy changes (e.g., API restrictions) making updated large-scale graph collection challenging.

- Explainability and coordinated behavior detection:
  While some models like multimodal GNNs and GPT-GNN frameworks begin to provide interpretable node/edge attributions, most GNN-based fake news detectors still act as black boxes.
  Coordinated inauthentic behavior (e.g., botnets, astroturfing) is usually not explicitly modeled as higher-order structures (e.g., dense near-cliques, repeated co-retweet patterns), leaving room for richer graph motif and subgraph pattern analysis.

Sachin:

1. Graph-Theory SEIR Health Misinformation in Aging Populations

- Weak empirical grounding of the model parameters:
  The SEIR transmission probabilities are said to be "calibrated with Bayesian inference," but the paper does not specify priors, likelihoods, or validation metrics, nor does it quantify uncertainty, leaving the realism of the diffusion dynamics under-argued.
- Insufficient dataset transparency and demographic validation:
  The Twitter/Facebook dataset for "aging" users is not described in terms of size, timeframe, labeling pipeline, or how age was inferred, so it is unclear how well the graphs actually represent older adults and how biased or noisy the data may be.
- Limited link between simulation outputs and real interventions:
  The paper simulates interventions (removing 5–10% of central nodes, literacy campaigns) but does not test these against real-world platform policies, sociotechnical constraints, or adversarial responses, leaving a large gap between model-based recommendations and operational deployment.
- Under-modeled heterogeneity in older adults and content types:
  Older adults are treated via generic parameters (trust, confirmation bias, fact-check frequency), with no modeling of subgroups by culture, language, comorbidities, or cognitive profiles, nor is health misinformation stratified into categories with distinct dynamics (acute epidemics vs chronic disease myths etc.).
- Ethics, privacy, and governance largely unaddressed:
  The work relies on large-scale social-media data about a vulnerable group but

does not engage with ethical review, data-protection constraints, or potential harms of “targeting” high-risk clusters, leaving a clear governance gap.

---

2. Knowledge-Based Fake News Detection with BERT + (Planned) GCN/GNN

- Core GCN/GNN contribution is not empirically realized:
  Although the title and methods emphasize combining BERT with GCN/GNN on a knowledge graph, the reported experiments only compare BERT and GPT-3 on a knowledge base; no actual GCN/GNN results or end-to-end graph-enhanced classifier are presented.
- Sparse description and release of the Twitter dataset:
  The Twitter corpus used to build the knowledge base is not characterized (N tweets, collection window, labeling protocol, topic balance), and is only available “on request,” which limits reproducibility and assessment of sampling and annotation bias.
- No ablation to justify SPO, sentiment, and topic components:
  SPO triples, SPO sentiment polarity, SPO occurrence, and LDA topics are added as knowledge-base attributes, but there is no systematic ablation to quantify how much each component contributes relative to plain BERT or GPT-3.
- Multimodal (image + text) side is only sketched:
  ResNet50-based image features and multimodal fusion are described conceptually, yet final experiments and metrics are purely textual; the model’s claims about Twitter images and multimodality remain untested.
- Scalability and streaming robustness not addressed:
  The framework is designed around offline knowledge-graph construction; there is no analysis of how it would handle high-velocity Twitter streams, frequent concept drift, adversarial perturbations, or near-real-time latency constraints.

---

3. Contextual + Sentiment + Social-Credibility Hybrid Fake News Detector

- Satire, borderline, and nuanced content remain problematic:
  The model frequently misclassifies satire (e.g., The Onion, Babylon Bee) and ambiguous news as fake due to reliance on exaggerated style and high sentiment polarity, indicating a gap in modeling pragmatic cues, intent, and genre distinctions.

- Multimodal misinformation (images, videos, memes) largely unaddressed:
  The system is text- and sentiment-centric with graph-based social features; it explicitly acknowledges that it cannot reliably handle image- and video-driven misinformation or meme-based campaigns, and calls for multimodal deep learning as future work.
- Limited exploration of explainability and human interpretability:
  The architecture is a complex ensemble (transformers, CNN, LSTM, GNN, RF, XGBoost), but no XAI tooling or explanation pipeline is implemented; the need for transparent, traceable decisions is deferred to future research.
- Generality across domains, languages, and platforms not validated:
  Experiments are on LIAR, FakeNewsNet, and BuzzFeedNews (mostly English and news-centric); performance on other domains (health, finance), non-English languages, and different platform dynamics is not studied.
- Deployment, robustness, and adversarial considerations are thin:
  There is no analysis of inference latency, scalability, or vulnerability to adversarial manipulation (e.g., style obfuscation, bot coordination changes), even though the model relies heavily on both stylistic and social-graph features.

Potential Datasets

- FibVID: Fake news diffusion dataset during the COVID-19 infodemic (Twitter)
    - Description: 1,353 news claims (True/False from Snopes, PolitiFact, etc.) with 221,253 related tweets/retweets from 144,741 users, including tweet IDs, user IDs (pseudonymized), propagation depths, and claim labels.
    - Graph aspects: Allows building claim-level diffusion trees, user–user retweet graphs, and user–claim bipartite graphs with temporal information.
    - Link:
    - https://pmc.ncbi.nlm.nih.gov/articles/PMC9759652/
    -
- MiDe22: Multi-Event Tweet Dataset for Misinformation Detection
    - Description: 10,348 tweets (English and Turkish) labeled as True/False/Other across 40 events (COVID-19, Russia–Ukraine war, refugees, etc.), plus user engagements (likes, replies, retweets, quotes) and media indicators.
    - Graph aspects: Engagement counts can be expanded into user–tweet and user–user interaction graphs if you fetch additional metadata via APIs or use provided IDs.
    - Link:
    - https://arxiv.org/html/2210.05401v2
    -
- COVID-19 Misinformation Twitter dataset (Misinformation_COVID-19)
    - Description: GitHub-hosted dataset of COVID-19 misinformation on Twitter, corresponding to an exploratory study; includes tweet IDs, labels, and metadata about misinformation categories.
    - Graph aspects: With tweet and user IDs, you can reconstruct retweet/mention/reply graphs via Twitter API (subject to current access limits) or use pre-collected interaction data if available.
    - Link:
    - https://github.com/Gautamshahi/Misinformation_COVID-19

Sachin:

1. FibVID – Fake news diffusion (COVID)

Zenodo (main download, CSV files):

https://zenodo.org/records/4441377

2. MuMiN (Multimodal Misinformation Social Network Dataset) - 2022

Best for: Heterogeneous graph construction and cross-event generalization.

How it works: Instead of simple user-user edges, MuMiN is an explicitly heterogeneous graph. It contains diverse node types: tweets, replies, users, images, articles, and hashtags.

https://colab.research.google.com/drive/1Kz0EQtySYQTo1ui8F2KZ6ERneZVf5TIN#scrollTo=5444ZX5rvrGL

3. UPFD (User Preference-aware Fake News Detection) - 2023

Best for: Out-of-the-box GNN baselines and rapid prototyping.

How it works: It provides tree-structured graphs based on fact-checks from PolitiFact and GossipCop. Node features are pre-encoded using BERT and spaCy, combined with 10-dimensional user profile features.

https://opendatalab.com/OpenDataLab/UPFD/tree/main/raw

- Need to login to the website to download the dataset

Eval Metrics

Evaluation metrics and baselines

Metrics commonly used

Empirical studies in GNN-based and propagation-based fake news detection report:

- Accuracy: Widely reported; hybrid and GNN-based methods often show gains of several percentage points over traditional baselines (e.g., ~97% accuracy for GNN hybrids in recent work).
- F1 score (macro / weighted): ATA-GNN and hybrid GNN models emphasize F1 to capture performance on both fake and real classes; reported F1 around 90–96% on Twitter15/16, PHEME, FakeNewsNet variants.
- ROC AUC: Used to assess ranking quality; ATA-GNN reports high AUC (93–97%) against prior graph-based baselines.
- Precision and recall for fake class: Often reported where fake content is the minority or of primary interest; some models highlight precision improvements (e.g., ATA-GNN achieving high precision on PHEME) and balanced recall.
- For rare or emerging events: AUCPR (area under precision–recall curve) and per-event F1 are recommended in survey discussions, especially when fake/news imbalance is severe.

For misinformation campaigns (rare coordinated behavior), explicitly reporting:

- Per-class precision/recall for "campaign-level" labels if you define them, and
- Detection delay (time to flag relative to first post)
  would be valuable extensions beyond standard metrics.

Evaluation matrix could include:

1. Text-only (e.g., BERT or RoBERTa fine-tuned on posts).
2. Network-only (graph features + RF/XGBoost).
3. Simple fusion (text features + graph features in an MLP).
4. Full GNN (heterogeneous GAT/GraphSAGE with text node features).

Phase 1 Questions

1. What are the datasets? Do u have them handy? The multiple datasets we have, are they related, how can we combine them?
2. The implementation, is it feasible?
3. Why are we doing this? When there are papers already doing this? Compare accuracy and if inefficient what are u doing to improve.
4. What is the objective in your mind?

Extra Info

This project develops a hybrid misinformation detection system that jointly models textual content and social propagation structure on social media. Text-only detectors struggle with nuanced or evolving narratives, while misinformation campaigns often exhibit characteristic diffusion patterns, such as rapid, shallow cascades or concentrated spread within tightly knit communities. To capture these effects, posts and users are represented in a graph where nodes correspond to content items and user accounts, and edges encode interactions such as retweets, replies, mentions, and shares. Each content node is initialized with high-dimensional features derived from a pre-trained language model (e.g., BERT embeddings), complemented by auxiliary signals such as sentiment polarity and emotion scores. In parallel, structural descriptors are computed from the interaction graph, including cascade depth and branching factor, user and post centrality measures, propagation speed over time, and community-level indicators of how widely and across which clusters a claim spreads. These structural features are either concatenated with content embeddings as node attributes in a graph neural network, or fused at a later stage in a classifier that operates on learned graph representations. The hybrid model thus learns how specific linguistic patterns interact with particular propagation signatures, enabling it to distinguish organically spreading true information from coordinated or anomalous misinformation cascades. Compared to traditional flat classifiers that operate on text or hand-crafted features in isolation, this design preserves relational context and diffusion dynamics, improving robustness to topic shift and enabling more interpretable analysis of which users, communities, and structural factors drive each detection decision.

Data Input Stage

- **Inputs**
    - Raw posts: tweet/news text, post IDs, timestamps, language.
    - User activity: user IDs, interactions (retweets, replies, mentions, quotes, likes), follower/followee relations if available.
    - Ground-truth labels at post/cascade level (fake vs real, or multi-class).
- **Processing steps**

    - Load CSV/JSON dumps from dataset (e.g., FakeNewsNet/FibVID/MiDe22) into tabular structures.

    - Align IDs across files (post ↔ user ↔ label).

- **Output format**

    - `pandas.DataFrame` objects: `posts_df`, `users_df`, `interactions_df`, `labels_df`.

- **Libraries**

    - `pandas`, `numpy`, `json`, `pyarrow` (optional for faster IO).pmc.ncbi.nlm.nih+1

Preprocessing Stage

- **Inputs**

    - `posts_df`, `users_df`, `interactions_df`.

- **Processing steps**

    - Text cleaning: lowercasing (if using uncased models), URL/user/hashtag normalization, optional language filtering.

    - Generate BERT-ready inputs: tokenize post text with HuggingFace tokenizer and store token IDs/attention masks; optionally precompute sentence embeddings via BERT and cache to disk.muegenai+2

    - Filter low-information or extremely short posts, and drop interactions with missing entities.

- **Output format**

    - Cleaned `posts_df` with columns: `post_id`, `user_id`, `text`, `timestamp`, `label`, `bert_input_ids`, `bert_attention_mask` or

`bert_emb` (if precomputed).

- **Libraries**
  - `pandas`, `re`, `transformers` (HuggingFace), `torch`.huggingface+1

Graph Construction Stage

- **Inputs**
  - Cleaned `posts_df`, `users_df`, `interactions_df`.
- **Processing steps**
  - Build a directed multigraph in NetworkX where nodes are users and posts; edges from interactions:
    - User → Post (authored, retweeted, replied, quoted, liked).
    - User → User (follows, mentions, co-retweet edges).
  - Attach basic node attributes: type (`user`/`post`), timestamps, label (for post nodes), and placeholder feature vectors (to be filled later).
  - Optionally construct per-claim or per-URL cascade subgraphs.
- **Output format**
  - `networkx.DiGraph` / `networkx.MultiDiGraph` for global graph and a list/dict of subgraphs for cascades.
- **Libraries**
  - `networkx`, `pandas`, `numpy`.codegenes+1

Feature Engineering Stage

- **Inputs**
    - Global interaction graph(s) (`networkx`), `posts_df` with BERT inputs/embeddings, `users_df`.
- **Processing steps**
    - **Content features:**
        - Run BERT (or similar) to obtain sentence-level embeddings for each post; map embeddings back to post nodes as `x_text`.github+2
        - Compute sentiment and emotion scores per post using an off-the-shelf model (e.g., transformer-based sentiment classifier) and attach as low-dimensional vectors.
    - **Structural features:**
        - Compute centrality measures (degree, in-degree, PageRank, betweenness, clustering coefficient) for user and post nodes.[pythontutorials]
        - For each cascade: compute depth, branching factor, temporal growth statistics (reshares vs time), structural virality.
        - For communities: run community detection (e.g., Louvain) on user–user or user–post projected graph; store community IDs and intra-/inter-community spread ratios.
    - Merge all features into node attribute dictionaries in NetworkX.
    - Convert NetworkX graphs to PyTorch Geometric `Data` objects using `from_networkx`, ensuring node features are stacked into `data.x`, labels into `data.y`, and edges into `data.edge_index` (and optionally `edge_attr`).apxml+4

- **Output format**
    - One or more `torch_geometric.data.Data` objects (or a custom `Dataset` subclass) with:
        - `x`: tensor `[num_nodes, num_features]` (concatenated text + sentiment/emotion + structural features).
        - `edge_index`: tensor `[2, num_edges]`.
        - `y`: node or graph labels (e.g., fake/real per post or cascade).
- **Libraries**
    - `networkx`, `torch`, `torch_geometric`, `transformers`, `pandas`, community detection libs (`python-louvain` or `networkx.algorithms.community`).pytorch-geometric.readthedocs+3

Model Training Pipeline

- **Inputs**
    - PyG `Data` objects (or `InMemoryDataset`) containing `x`, `edge_index`, `y`, and optional train/val/test masks.
- **Processing steps**
    - Split data into train/validation/test sets, either at node level (mask tensors) or graph level (dataset indices).
    - Define GNN model (e.g., GCN, GraphSAGE, or GAT) that consumes `x` and `edge_index` and outputs logits for each labeled node/graph. The model implicitly learns from the concatenated content+structural features.

    - Implement training loop: forward pass, supervised loss (cross-entropy), backpropagation, optimizer step, early stopping based on validation performance.

    - Optionally, train baselines (text-only BERT classifier, RF/XGBoost on engineered features) in parallel for comparison.

- **Output format**

    - Trained PyTorch model checkpoints (`.pt`), along with logs of metrics (accuracy, F1, ROC AUC, per-class precision/recall) stored in JSON/CSV.

- **Libraries**

    - `torch`, `torch_geometric`, `sklearn` (for metrics & RF/XGBoost wrappers), `xgboost` (optional), `transformers`.

Inference Pipeline

- **Inputs**

    - New raw posts and user activity (same schema as training), trained model checkpoints, preprocessing/feature pipelines.

- **Processing steps**

    - Apply the same preprocessing: text cleaning, tokenization/BERT embedding, sentiment/emotion scoring.

    - Incrementally update or construct the relevant graph segment (e.g., latest cascades or local ego-graph) and convert to PyG `Data` object with consistent feature layout.

    - Load trained GNN model and run forward pass to obtain predicted probabilities or logits for each target node/graph.

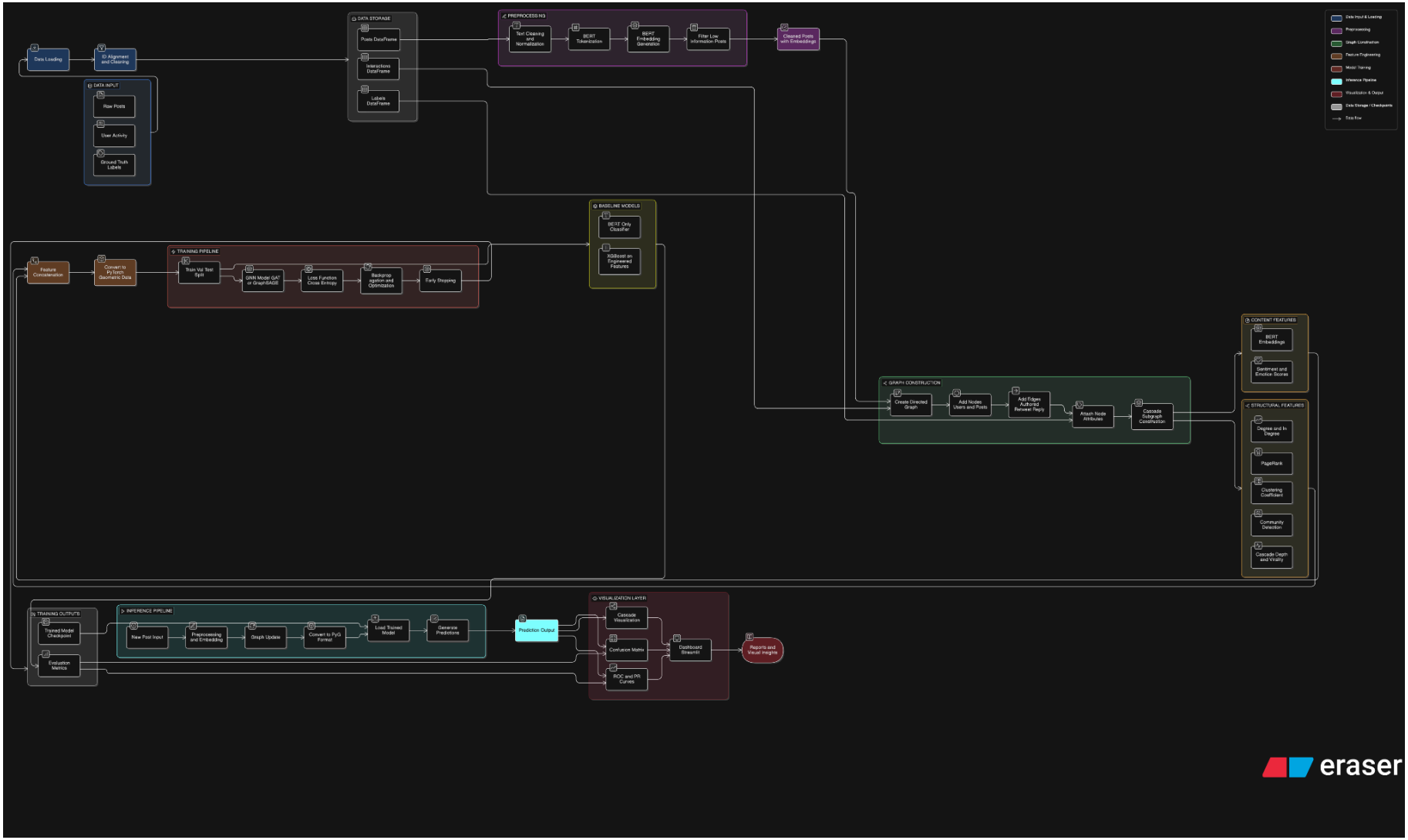
- **Output format**
    - Predictions as `pandas.DataFrame` with columns: `post_id`, `pred_label`, `pred_prob_fake`, `pred_prob_real`, plus any auxiliary explanation scores if computed.
- **Libraries**
    - Same as training pipeline (`pandas`, `torch`, `torch_geometric`, `transformers`, `networkx`).

Output & Visualization Layer

- **Inputs**
    - Prediction dataframe, graph structures, trained model introspection (e.g., node importance scores).
- **Processing steps**
    - Generate evaluation plots: ROC curves, precision–recall curves, confusion matrices using `matplotlib`/`seaborn` and `sklearn.metrics`.
    - Visualize selected cascades with NetworkX: color nodes by predicted label and centrality, edge thickness by propagation time or weight.
    - Optionally integrate basic dashboard (e.g., `Streamlit` or `Dash`) to explore individual cascades, user roles, and model predictions.
- **Output format**
    - Static figures (`.png`), notebooks (`.ipynb`), or lightweight web dashboard; human-readable reports summarizing performance and qualitative case studies.
- **Libraries**

- `matplotlib`, `seaborn`, `sklearn.metrics`, `networkx`, `streamlit`/`dash` (optional).

Flowchart


Data Loading
ID Alignment and Cleaning
DATA INPUT
Raw Posts
User Activity
Ground Truth Labels
DATA STORAGE
Posts DataFrame
Interactions DataFrame
Labels DataFrame
PREPROCESSING
Text Cleaning and Normalization
BERT Tokenization
BERT Embedding Generation
Filter Low Information Posts
Cleaned Posts with Embeddings
BASELINE MODELS
BERT Only Classifier
XGBoost on Engineered Features
Feature Concatenation
Convert to PyTorch Geometric Data
TRAINING PIPELINE
Train Val Test Split
GRN Model GAT or GraphSAGE
Loss Function Cross Entropy
Backpropagation and Optimization
Early Stopping
GRAPH CONSTRUCTION
Create Directed Graph
Add Nodes Users and Posts
Add Edges Authored Retweet Reply
Attach Node Attributes
Cascade Subgraph Construction
CONTENT FEATURES
BERT Embeddings
Sentiment and Emotion Scores
STRUCTURAL FEATURES
Degree and In Degree
PageRank
Clustering Coefficient
Community Detection
Cascade Depth and Virality
TRAINING OUTPUTS
Trained Model Checkpoint
Evaluation Metrics
INFERENCE PIPELINE
New Post Input
Preprocessing and Embedding
Graph Update
Convert to PyG Format
Load Trained Model
Generate Predictions
Prediction Output
VISUALIZATION LAYER
Cascade Visualization
Confusion Matrix
ROC and PR Curves
Dashboard Streamlit
Reports and Visual Insights
Data Input & Loading
Preprocessing
Graph Construction
Feature Engineering
Model Training
Inference Pipeline
Visualization & Output
Data Storage / Checkpoints
Data Flow